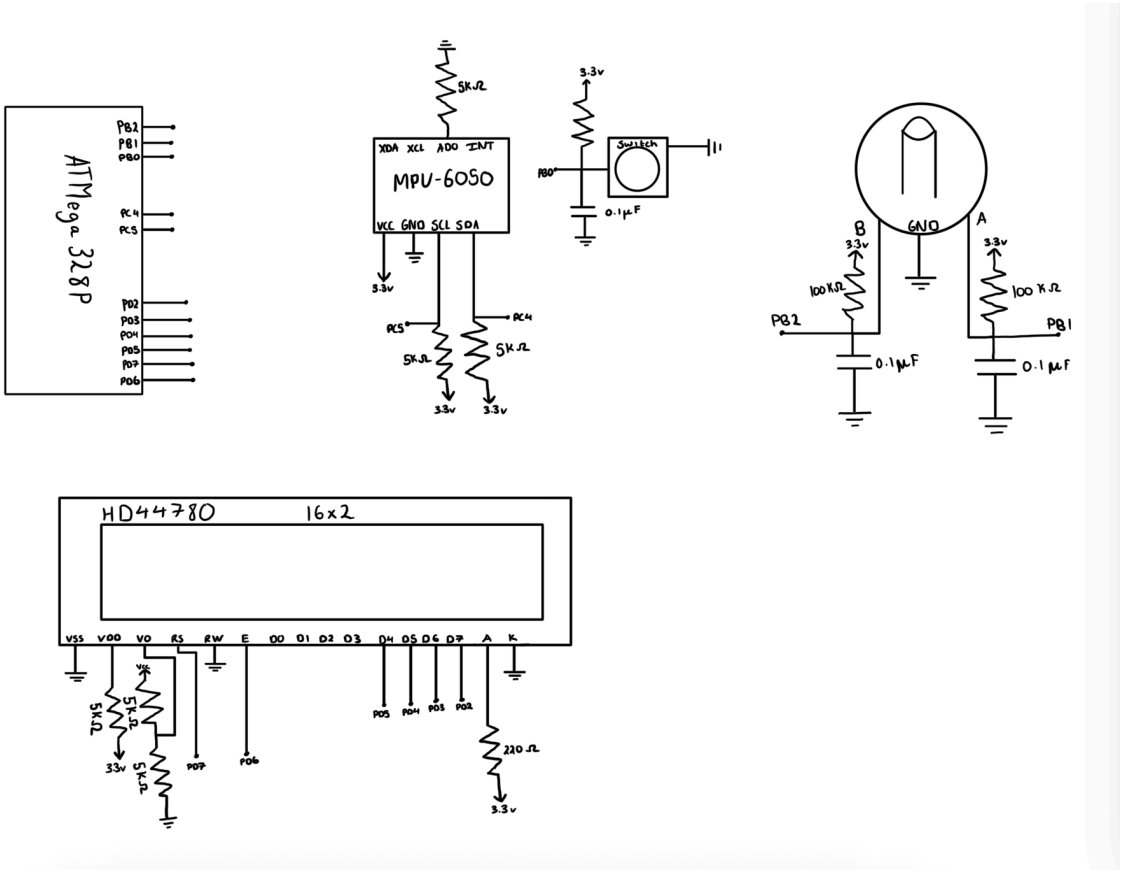
Player vs AI Pong Game
Group 5: Tallen Lawrence, Kyan Shepherd
Fall 2025
ECE:3360 Embedded Systems

Introduction:

Our motivation behind this project was to take an old arcade game, in this case pong, and add a more advanced modern touch to it. Here the idea was to create a controller that utilised level detecting to move the pong paddle, we also aimed to have an ai opponent with selectable difficulty to ensure the user can try new challenges. We also wanted the controller to be as interactive as possible, so we achieved this by adding a PBS and RPG to allow even more user input and for the system to be fully controllable from just the controller alone.

Implementation:



To discuss the big picture here, the end goal was to have two main subsystems that control the two parts of the game. One subsystem would be the controller, this would be responsible for all the inputs regarding the game. The second would be the display, this would be responsible for displaying the game, the interface and all interactions the player can have with the controller. So regarding the controller side the hardware components we are using are an MPU 6050 gyroscope, a push button switch and a rotary pulse generator. These would all be on the same

breadboard with the gyroscope in the middle and the switch and rpg on opposite sides to reflect how a normal games controller would be. The RPG was responsible for all the selectable options, for example, if you want to change the game mode you can rotate the RPG to change the difficulty or when the game is paused you can rotate the RPG to decide whether to resume or exit. The RPG was not used at a;l regarding the movement of the paddle or any other part of the interface. The PBS was used as a selection method after the RPG input had been decided, the game would ask for you to press the button to begin first, once it was registered as pressed we would then ask for a difficulty to be selected, the PBS would confirm the difficulty when pressed and start the game. During the game if the PBS was held for 2 or more seconds the pause menu would open and the options provided would be resume or exit. If you chose to resume you would press the PBS again to continue the game, or if you chose to exit you would press the PBS to also confirm your choice and the user would return to the screen asking them to press the button to start again and the game resets. Then regarding the display subsystem we used a HD44780 16x2 LCD display to display the game and all the menu options. Regarding the game display the screen was split into thirds, 2 thirds were used to just display the game on the left side of the display, the other third was used to display an interactive scoreboard. The LCD was connected directly to the micro controller and this formed the base of the project . so all components were connected directly to the micro controller and controlled this way. The RPG input could only be valid if the state of the game was in a menu with a selectable part of the game, and the only way to advance was to press the PBS . so the design process simply started with the idea of a rotating controller and a pong game, we wanted a level detecting controller similar to a WII remote to be able to play a game on an LCD all in one system. We didn't want to use an external game engine and wanted the controller to be realistic with buttons and an RPG for better user customisation regarding the game, also most game controllers have a similar concept included. So regarding the component selection , we chose to use a HD44780 LCD this is because originally we had planned to use a graphical display , like one found in a nokia phone, however due to distribution issues this did not arrive in time so we had to improvise and use something we had used prior to this and were comfortable with, it also had all the functionality we needed with custom characters allowing us to split the screen into even smaller parts to allow for better game real estate on the display. When looking for something for level detecting , a gyroscope was the best option here, cheap and reliable. We chose the MPU 6050 because it does a great job at balancing cost and reliability, it also includes a 3-axis gyroscope and a 3-axis accelerometer which we used to implement tilt detection, also it worked on both a 3.3V and 5V power supply. This was great as we did not need to use level changing when combining the two subsystems taken from the micro controller. Also, we did not require a very high resolution as we used tolerances to balance this issue out, therefore this was the best selection we could find. Now, regarding the software, we began by setting up the easier parts, so we used C to code a loop that recognised when a button was pressed and printed continue in the serial monitor, we also did this to test our debouncing circuit and see how well it stopped spikes in the input. Once we had this we began to develop the interface, using strings to print words on the LCD we make it so it would loop an original routine of displaying press button to start game until a button press was detected , we then jumped to the difficulty selection mode which we then implemented the RPG , using previous code from labs, to detect a CW or CCW rotation and then change the difficulty mode using strings to print when assigned to the value of

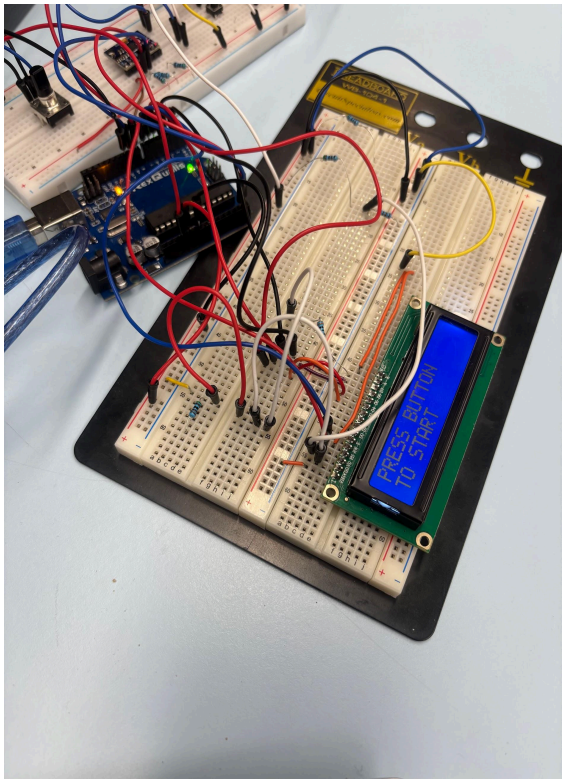
rotation of the RPG. Then we looped the button routine again so the game could only start if the button was pressed. Then regarding the game itself, we used custom characters on the LCD to map the paddles , the borders, and strings to map the score board. Regarding the ball and its physics, we developed a system where we would flip its velocity and measure its x axis and y axis and predict where it would then move allowing for diagonal and more realistic physics. When the walls were hit we also flipped its y velocity and the paddle would flip its x velocity. After a point was scored , the ball would go to the middle, delay a second so the player could respond and then move toward the player who just conceded the point. We also used ISR regarding the pause menu; if the button was held for 2+ seconds during the game, values would be saved and the game would stop and open a pause menu of which we can resume, return to the previous code, or exit, reset the system and go back to the start. Then once a player had won, the game displayed if they won or lost for 7 seconds and then reset the system and returned back to the original menu, waiting for a switch input. Regarding the gyroscope, we measured the pitch and converted this into an angle we could read and change the value of a tilt register to save where the paddle should be, we also added a threshold so the tilt value wouldn't change unless a degree of +- 5 degrees was met , to reduce errors when it came to level readings fluctuating due to noise in the gyroscope.

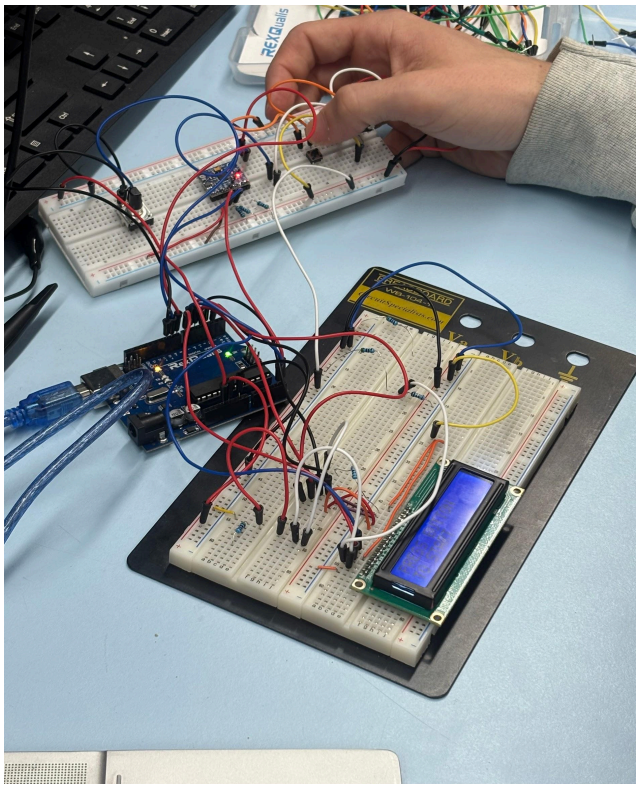
Experimental Methods:

For this project, we divided and conquered the workload. Each of us handled and tested our components individually before combining them into one system. This gave us an efficient workflow and let us implement and test each part without overlap. To set up the LCD, we started with a simple “hello world” program to confirm the hardware was wired correctly. We tested the backlight using an oscilloscope, measuring the voltage across resistors until we achieved the desired brightness. Once the hardware was set, we moved on to the software. Using the Arduino IDE, we compiled and debugged our code, and with the LCD library we learned how to use custom characters to design our game sprites. For the gyroscope, we used the Arduino serial monitor to test the controller and understand its behavior. After both components were working properly, we integrated them into a single embedded system. From there, we tied the gyroscope input into the game, mapping paddle movement to half of the LCD screen (positions 0–7) and defining the ball’s movement on a 4×16 area. Next, we implemented the game states, allowing the user to choose difficulty, pause, and exit. Once that flow worked smoothly, we developed the bot behavior, added the win condition, and refined the overall game logic until we were satisfied with the final result.

Results:

Regarding the results, there was not much we could test here other than final game functionality. In the image provided below you can see a full picture of the final hardware wired up including the controller breadboard with wires connecting to the ATMega328P and also the display breadboard with our LCD. you can also see the interface showing the correct start message “PRESS BUTTON TO START”. Here we are showcasing the final product we have created and showing some of its functionality being displayed on the LCD display. We used the serial monitor in the arduino to check the threshold results we had set to limit the gyroscopes error margin by making a tilt forward or backward only count after a certain tilt value, this was because the gyroscope when set at level would constantly change its tilt value randomly , so to stop this noise affecting the reading we made it so it would only register after a certain tilt level was reached so when it was level it would never move the paddle left or right. We also demonstrated full functionality with every feature and aspect included during our project video we provided , showing how the controller works, how the display works, and how the interface was set up.





Discussion of Results:

To begin to discuss results we will answer the questions provided, so how well does the device perform? compared to our specification i believe the device performs very well, the tilt has threshold values to stop misreading's and to help combat the noise interference. As the user rotates the controller forward or backward the degree of incline perfectly corresponds to a change in movement on the paddle, with the steeper the

angle correlating to a faster movement. The menu and interface utilize the other components of the controller really well with a smooth running system and even a pause menu for user experience. I think this provides a really good user experience with the controller solely controlling the game/menus. The LCD smoothly updates and allows for a smooth ball movement and paddle interaction system which overall performs really well considering the circumstances of the 16x2 LCS display being a last resort. Overall the device performs very well in comparison to our original idea and specification. Now to discuss if the design goals were met. So originally we planned to use a graphical LCD in the plan, however, due to distribution issues, this was no longer an option. Instead we took the next best option of the 16x2 LCD we had used prior. I believe this is the only design goal that was not met in reference to the original specification, but through improvisation we made it work well with the smaller 16x2 LCD display. The controller design goals were simple and met perfectly, with level detecting implemented, also using a mixture of PBS input and RPG input to control and navigate the menus. We were really happy with how the controller turned out. We originally would've liked to use a Bluetooth connection, however this was too advanced in the time frame we had so to improvise we simply used extended wires so the user can hold the controller freely with no obstacles with rotation or distance from the ATMega328P. Now the implementation alternatives, we had one main one which was the LCD. Originally we wanted to use a graphical display used in Nokia phones , this would've allowed for a faster refresh rate and in turn a smoother gameplay. However we improvised when we knew it wouldn't arrive in time, we used what we were familiar with, the 16x2 LCD display. It was a struggle to use custom characters regarding the ball interaction with the paddle , however in the end we got this sorted. Other than this we wished to apply a Bluetooth connection between the controllers, but after understanding the workload we realized this would be too much for the time frame so we opted for longer wires for more freedom of movement. Current limitations include the visual real estate of the 16x2 LCD and its refresh rate, making it a lot harder for a smoothly running game, also the noise from the MPU 6050 was a challenge, to tackle this we created thresholds of when the game would register a rotation as noise created small variations in readings when the sensor was flat or slightly rotated. This worked very well. The ideas for improvement include a Bluetooth connection between boards to allow for 0 obstacles in the controller movement. we would've also liked to print a case for the controller to allow for a smooth user experience in terms of handling and gameplay, also a faster refreshing and larger LCD would've allowed for the game to run smoother and more realistic to the original game.

Conclusion:

In this project we combined different aspects of software and hardware implementation to develop a fully self sustaining game and game controller system. It would take inputs from 3 different components that all would influence either the game or user interface. We wanted a level detecting controller and a programmed pong game on an LCD display that a user could play again and again with different difficulties to maintain a fun and exciting experience throughout. This could be developed further with a 3d printed controller base to help a more user friendly experience and we would have liked to implement a graphical display for a smoother user experience, however due to distribution issues, this was not a possibility for us. This was a very fun and interactive project that we both enjoyed working on combining our interest in video games and controller design.

References:

MPU 6050 Data Sheet
InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification*, Rev. 3.4, San Jose, CA, USA, Aug. 2013

Source Code:

```
#define F_CPU 16000000UL
#include <Arduino.h>
#include <LiquidCrystal.h>
#include <math.h>

// MPU/TWI / pins
#define MPU6050_ADDR 0x68
#define BUTTON_PIN PB0
#define RPG_A PB1
#define RPG_B PB2

#define PAUSE_HOLD_MS 1000
#define BUTTON_DEBOUNCE_MS 50

// LCD (RS, E, D4, D5, D6, D7)
LiquidCrystal lcd(7, 6, 5, 4, 3, 2);

// ---- custom chars ----
  byte dude[8] = {
  0b01110,
  0b01110,
```

```
  0b00100,
  0b11111,
  0b00100,
  0b00100,
  0b01010,
  0b10001
};

  byte paddle_bottom[8] = {
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b11111
};

  byte paddle_top[8] = {
  0b11111,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000
};


byte blank[8] = {
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000
};

byte gamewall[8] = {
  0b10000,
```

```
  0b10000,
  0b10000,
  0b10000,
  0b10000,
  0b10000,
  0b10000,
  0b10000
};

 byte ball_topl[8] = {
  0b11100,
  0b11100,
  0b11100,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000
};

 byte ball_topm[8] = {
  0b01110,
  0b01110,
  0b01110,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000
};

 byte ball_topr[8] = {
  0b00111,
  0b00111,
  0b00111,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000
};

 byte ball_m[8] = {
  0b00000,
```

```
  0b00000,
  0b01110,
  0b01110,
  0b01110,
  0b00000,
  0b00000,
  0b00000
};

 byte ball_botl[8] = {
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b11100,
  0b11100,
  0b11100
};

 byte ball_botm[8] = {
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b01110,
  0b01110,
  0b01110
};

 byte ball_botr[8] = {
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00111,
  0b00111,
  0b00111,
};

byte pause[8] = {
  0b11011,
```

```
  0b11011,
  0b11011,
  0b11011,
  0b11011,
  0b11011,
  0b11011,
  0b11011,

};
 byte bottom_paddle_ball_botl[8] = {
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b11100,
  0b11100,
  0b11111,
};
 byte bottom_paddle_ball_botr[8] = {
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00111,
  0b00111,
  0b11111,
};

 byte bottom_paddle_ball_topl[8] = {
  0b11100,
  0b11100,
  0b11100,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b11111,
};
 byte bottom_paddle_ball_topr[8] = {
  0b00111,
  0b00111,
  0b00111,
```

```
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b11111,
};
 byte top_paddle_ball_botl[8] = {
  0b11111,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b11100,
  0b11100,
  0b11100,
};

 byte top_paddle_ball_botr[8] = {
  0b11111,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00111,
  0b00111,
  0b00111,
};

 byte top_paddle_ball_topl[8] = {
  0b11111,
  0b11100,
  0b11100,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
  0b00000,
};

 byte top_paddle_ball_topr[8] = {
  0b11111,
  0b00111,
  0b00111,
  0b00000,
```

```
  0b00000,
  0b00000,
  0b00000,
  0b00000,
};

// ------------------ Minimal tile paging additions ------------------
// Tile IDs (logical identifiers)
#define TILE_DUDE              0
#define TILE_PADDLE_BOTTOM       1
#define TILE_PADDLE_TOP          2
#define TILE_GAMEWALL            3
#define TILE_BALL_TOPL           4
#define TILE_BALL_TOPM           5
#define TILE_BALL_TOPR           6
#define TILE_BALL_M              7
#define TILE_BALL_BOTL           8
#define TILE_BALL_BOTM           9
#define TILE_BALL_BOTR          10
#define TILE_BOTTOM_P_TOPL      11
#define TILE_BOTTOM_P_TOPR      12
#define TILE_BOTTOM_P_BOTL      13
#define TILE_BOTTOM_P_BOTR      14
#define TILE_TOP_P_TOPL         15
#define TILE_TOP_P_TOPR         16
#define TILE_TOP_P_BOTL         17
#define TILE_TOP_P_BOTR         18
#define TILE_PAUSE_ICON         19


byte* tilePointer(int tileID) {
  switch(tileID) {
    case TILE_DUDE: return dude;
    case TILE_PADDLE_BOTTOM: return paddle_bottom;
    case TILE_PADDLE_TOP: return paddle_top;
    case TILE_GAMEWALL: return gamewall;
    case TILE_BALL_TOPL: return ball_topl;
    case TILE_BALL_TOPM: return ball_topm;
    case TILE_BALL_TOPR: return ball_topr;
    case TILE_BALL_M: return ball_m;
    case TILE_BALL_BOTL: return ball_botl;
    case TILE_BALL_BOTM: return ball_botm;
    case TILE_BALL_BOTR: return ball_botr;
    case TILE_BOTTOM_P_TOPL: return bottom_paddle_ball_topl;
```

```
    case TILE_BOTTOM_P_TOPR: return bottom_paddle_ball_topr;
    case TILE_BOTTOM_P_BOTL: return bottom_paddle_ball_botl;
    case TILE_BOTTOM_P_BOTR: return bottom_paddle_ball_botr;
    case TILE_TOP_P_TOPL: return top_paddle_ball_topl;
    case TILE_TOP_P_TOPR: return top_paddle_ball_topr;
    case TILE_TOP_P_BOTL: return top_paddle_ball_botl;
    case TILE_TOP_P_BOTR: return top_paddle_ball_botr;
    case TILE_PAUSE_ICON: return pause;
    default: return blank;
  }
}

// Tracks what tile ID is currently stored in each CGRAM slot
// -1 means slot unused.
int loadedTile[8] = { -1, -1, -1, -1, -1, -1, -1, -1 };

// Load logical tile (tileID) into a CGRAM slot if not already loaded.
// Minimal policy: use first free slot; if none free overwrite slot 7.
void loadTile(int tileID) {
  // If already loaded, nothing to do.
  for (int i = 0; i < 8; ++i) {
    if (loadedTile[i] == tileID) return;
  }

  // Find a free slot (prefer slots that aren't used)
  int slot = -1;
  for (int i = 0; i < 8; ++i) {
    if (loadedTile[i] == -1) { slot = i; break; }
  }
  if (slot == -1) {
    // If no free slot, overwrite slot 7 (minimal policy)
    slot = 7;
  }

  // write CGRAM
  lcd.createChar(slot, tilePointer(tileID));
  loadedTile[slot] = tileID;
}

// Return CGRAM slot index where tileID is loaded, -1 if not present
int tileSlot(int tileID) {
  for (int i = 0; i < 8; ++i) {
    if (loadedTile[i] == tileID) return i;
  }
```

```
  return -1;
}

// ------------------ Game state ------------------
static int cursorValue = 4;   // paddle column (0..15)
static int botCursorValue = 4;
static int currentX = 8;      // ball pixel-x (0..15*2 if using half-cell) - we will use 0..15*2
static int currentY = 0;      // ball pixel-y (0..15*2 mapping to 2 rows -> here we use 0..15
effectively)
static int ballDX = 0;  // horiztonal direction: +1 right, 0 static, -1 left
static int ballDY = 1; // vertical direction: +1 down, -1 up
static int playerScore = 0;
static int botScore = 0;
static int botPD = 1; //bottle paddle direction (for easy and medium modes)
static int timeInterval = 0;

// timing
unsigned long lastBallMove = 0;
const unsigned long BALL_INTERVAL_MS = 1000; // ball moves every 300ms (tweakable)

// ----------------- Forward declarations -----------------
void UART_Init(void);
void UART_TxString(const char* s);
void TWI_Init(void);
void TWI_Start(void);
void TWI_Stop(void);
uint8_t TWI_Write(uint8_t data);
uint8_t TWI_Read_ACK(void);
uint8_t TWI_Read_NACK(void);
void MPU6050_Init(void);
void MPU6050_ReadAll(int16_t* ax,int16_t* ay,int16_t* az,int16_t* gx,int16_t* gy,int16_t* gz);
uint8_t readButton(void);
void waitForButtonPress(void);
uint8_t chooseDifficultyRPG(void);
uint8_t pauseMenu(void);
void displayStart(int mode);
void displayPause(int option);
void displayGame(int p1, int p2);
void handleBall(void);
int drawBall(void);

// ----------------- Setup -----------------
void setup() {
```

```
  Serial.begin(9600);
  while(!Serial) { ; } // wait for Serial (optional on some boards)

  // LCD init
  lcd.begin(16, 2);
  //lcd.createChar(0, dude);
  lcd.createChar(1, paddle_bottom);
  lcd.createChar(2, paddle_top);
  lcd.createChar(3, gamewall);
  lcd.createChar(4, ball_topl);//dynamic 4-7
  lcd.createChar(5, ball_topr);
  lcd.createChar(6, ball_botl);
  lcd.createChar(7, ball_botr);
 // lcd.createChar(8, pause);
  //lcd.createChar(9,bottom_paddle_ball_botl);

  // initialize loadedTile[] to match the above chars

  for (int i = 0; i < 8; ++i) loadedTile[i] = -1;
  loadedTile[1] = TILE_PADDLE_BOTTOM;
  loadedTile[2] = TILE_PADDLE_TOP;
  loadedTile[3] = TILE_GAMEWALL;
  loadedTile[4] = TILE_BALL_TOPL;
  loadedTile[5] = TILE_BALL_TOPR;
  loadedTile[6] = TILE_BALL_BOTL;
  loadedTile[7] = TILE_BALL_BOTR;


  // initialize TWI and MPU
  TWI_Init();
  MPU6050_Init();

  // set button pin as input with pull-up
  DDRB &= ~(1<<BUTTON_PIN);
  PORTB |= (1<<BUTTON_PIN);

  // init state
  cursorValue = 4;
  botCursorValue = 4;
  currentX = 8;    // center-ish
  currentY = 0;    // start top
  ballDX = 0; //ball direction x (left and right)
  ballDY = 1; //ball direction y (up and down)
  playerScore = 0;
```

```
  botScore = 0;
  botPD = 1;
  timeInterval = 0;

  displayStart(); // show difficulty prompt
}

// ----------------- Loop (main game flow) -----------------
void loop() {
  // Use Serial wrapper for backward compatibility with UART_TxString calls
  // (the implementation below uses Serial)

  // Wait for start
  UART_TxString("Press button to start game...\r\n");
  _delay_ms(200);
  waitForButtonPress();

  displayDifficulty(1);

  int mode = chooseDifficultyRPG();
  UART_TxString("Game starting...\r\n");
  _delay_ms(500);

  // start game loop
  cursorValue = 4; // reset paddle
  botCursorValue = 4;
  currentX = 8;
  currentY = 0;
  ballDX = 0;
  ballDY = 1;
  botPD = 1;
  timeInterval = 0;

  //game in progress
  while (playerScore != 11 && botScore != 11) {

    //_delay_ms(500);
    // read MPU6050
    int16_t ax,ay,az,gx,gy,gz;
    MPU6050_ReadAll(&ax,&ay,&az,&gx,&gy,&gz);
    float axg = ax/16384.0f;
    float ayg = ay/16384.0f;
    float azg = az/16384.0f;
    float pitch = atan2f(axg, sqrtf(ayg*ayg + azg*azg)) * 180.0f / 3.14159265f;
```

```
// Paddle movement smoothing
    static unsigned long lastPaddleMove = 0;       // remembers last move time
    const unsigned long PADDLE_INTERVAL_MS = 250; // move paddle only every 250ms

    int8_t paddlePosition = cursorValue; // local temp
    unsigned long now = millis();

// Only update paddle if enough time has passed
    if (now - lastPaddleMove >= PADDLE_INTERVAL_MS) {
    lastPaddleMove = now;

    // Only move if tilt is significant
        if (pitch > 20.0f && paddlePosition < 7) paddlePosition++;
        else if (pitch < -20.0f && paddlePosition > 0) paddlePosition--;
}

// commit paddle
cursorValue = paddlePosition;

    //update botPaddlePosition
    handleBot(mode);

    // debug print
    char buffer[64];
    dtostrf(pitch,6,2,buffer);
    UART_TxString("Pitch = ");
    UART_TxString(buffer);
    sprintf(buffer," | Game paddle = %d\r\n", paddlePosition );
    UART_TxString(buffer);

    // Handle pause button (hold)
    if (readButton()) {
      // measure hold time
      uint16_t hold = 0;
      while (readButton()) {
        _delay_ms(50);
        hold += 50;
      }
      if (hold >= PAUSE_HOLD_MS) {
        _delay_ms(100);
        uint8_t choice = pauseMenu();
        if (choice == 1) {
          UART_TxString("Resuming game...\r\n");
          // continue game
```

```
      } else {
        UART_TxString("Exiting to start...\r\n");
        _delay_ms(300);
        handleRestart();
        handleExit();
        displayStart();
        break; // exit to outer start loop
      }
    }
  }

  // Move ball at interval
  //unsigned long now = millis();
  if (now - lastBallMove >= BALL_INTERVAL_MS) {
    lastBallMove = now;
    handleBall(); // updates currentY and handles scoring / bouncing
  }

  // Render game to LCD
  displayGame(cursorValue, botCursorValue);


  _delay_ms(50);
 } // end inner game loop
  if(playerScore == 11){
    handleWin(1);
  }
  else if(botScore == 11){
    handleWin(2);
  }
  displayStart();
 // loop will return to start prompt
}

// ----------------- UART wrapper (uses Serial) -----------------
void UART_Init(void) {
 // For compatibility: ensure Serial already started in setup()
 if (!Serial) Serial.begin(9600);
}
void UART_TxString(const char* s) {
 Serial.print(s);
}

// ----------------- TWI / I2C code -----------------
```

```
void TWI_Init(void){ TWSR=0; TWBR=72; }
void TWI_Start(void){ TWCR=(1<<TWINT)|(1<<TWSTA)|(1<<TWEN); while(!(TWCR &
(1<<TWINT))); }
void TWI_Stop(void){ TWCR=(1<<TWINT)|(1<<TWEN)|(1<<TWSTO); }
uint8_t TWI_Write(uint8_t data){ TWDR=data; TWCR=(1<<TWINT)|(1<<TWEN); while(!(TWCR
& (1<<TWINT))); return (TWSR & 0xF8); }
uint8_t TWI_Read_ACK(void){ TWCR=(1<<TWINT)|(1<<TWEN)|(1<<TWEA); while(!(TWCR &
(1<<TWINT))); return TWDR; }
uint8_t TWI_Read_NACK(void){ TWCR=(1<<TWINT)|(1<<TWEN); while(!(TWCR &
(1<<TWINT))); return TWDR; }

// MPU6050 init/read (keeps your partner's implementation)
void MPU6050_Init(void){ TWI_Start(); TWI_Write(MPU6050_ADDR<<1);TWI_Write(0x6B);
TWI_Write(0x00); TWI_Stop(); }
void MPU6050_ReadAll(int16_t* ax,int16_t* ay,int16_t* az,int16_t* gx,int16_t* gy,int16_t* gz){
    uint8_t data[14];
    TWI_Start(); TWI_Write(MPU6050_ADDR<<1); TWI_Write(0x3B);
    TWI_Start(); TWI_Write((MPU6050_ADDR<<1)|1);
    for(int i=0;i<13;i++) data[i]=TWI_Read_ACK();
    data[13]=TWI_Read_NACK();
    TWI_Stop();
    *ax=(data[0]<<8)|data[1]; *ay=(data[2]<<8)|data[3]; *az=(data[4]<<8)|data[5];
    *gx=(data[8]<<8)|data[9]; *gy=(data[10]<<8)|data[11]; *gz=(data[12]<<8)|data[13];
}

// ----------------- Button / input helpers -----------------
uint8_t readButton(void){
    if(!(PINB & (1<<BUTTON_PIN))){
        _delay_ms(BUTTON_DEBOUNCE_MS);
        if(!(PINB & (1<<BUTTON_PIN))) return 1;
    }
    return 0;
}
void waitForButtonPress(void){
    while(!readButton());
    while(readButton());
}

// ----------------- RPG / Difficulty selection -----------------
uint8_t chooseDifficultyRPG(void){
    DDRB &= ~((1<<RPG_A)|(1<<RPG_B));
    PORTB |= (1<<RPG_A)|(1<<RPG_B);

    uint8_t lastA = (PINB & (1<<RPG_A)) ? 1 : 0;
```

```
    uint8_t difficulty = 1; // Easy default

    UART_TxString("Rotate RPG to change difficulty, press button to confirm...\r\n");
    UART_TxString("Current mode: Easy\r\n");

    while(1){
        uint8_t currentA = (PINB & (1<<RPG_A)) ? 1 : 0;
        if(lastA && !currentA){ // falling edge
            if(PINB & (1<<RPG_B)){
                if(difficulty < 3) difficulty++;
            } else {
                if(difficulty > 1) difficulty--;
            }
            UART_TxString("Current mode: ");
            if(difficulty==1){
              UART_TxString("Easy\r\n");
              displayDifficulty(difficulty);
            }
            else if(difficulty==2) {
              UART_TxString("Medium\r\n");
              displayDifficulty(difficulty);
            }
            else {
              UART_TxString("Hard\r\n");
              displayDifficulty(difficulty);
            }
            _delay_ms(200);
        }
        lastA = currentA;

        if(readButton()){
            _delay_ms(200); // debounce
            //set difficulty to selected difficulty
            UART_TxString("Difficulty confirmed: ");
            if(difficulty==1)  UART_TxString("Easy\r\n");
            else if(difficulty==2) UART_TxString("Medium\r\n");
            else UART_TxString("Hard\r\n");
            _delay_ms(500);
            return difficulty;
        }
    }
}

// ----------------- Pause menu -----------------
```

```
uint8_t pauseMenu(void){
    DDRB &= ~((1<<RPG_A)|(1<<RPG_B));
    PORTB |= (1<<RPG_A)|(1<<RPG_B);

    UART_TxString("\r\n--- PAUSE MENU ---\r\n");
    uint8_t lastA = (PINB & (1<<RPG_A)) ? 1 : 0;
    uint8_t selection = 1; // 1=Resume, 2=Exit
    UART_TxString("Selected: Resume\r\n");
    displayPause(selection);

    while(1){
        uint8_t currentA = (PINB & (1<<RPG_A)) ? 1 : 0;
        if(lastA && !currentA){
            if(PINB & (1<<RPG_B)){
                if(selection < 2) selection++;
            } else {
                if(selection > 1) selection--;
            }
            UART_TxString("Selected: ");
            if(selection==1) UART_TxString("Resume\r\n");
            else UART_TxString("Exit\r\n");
            _delay_ms(200);

            displayPause(selection);
        }
        lastA = currentA;

        if(readButton()){
            _delay_ms(200); // debounce
            return selection; // return choice
        }
    }
}

// ----------------- Ball handling -----------------
void handleBall(){
  // simple vertical movement and collision with paddle (cursorValue)
  // currentX/currentY represent "half-pixel" coordinates mapped to chars by /2 in display
  int nextX = currentX + ballDX;
  int nextY = currentY + ballDY;

//wall collisions
  if (nextX < 0) {
        ballDX = 1;      // bounce right
```

```
        nextX = 1;
    }
    else if (nextX > 15) {
        ballDX = -1;     // bounce left
        nextX = 15;
    }

//handle scores
  //top wall (bot goal, player scores)

    if (nextY < 0) {
        playerScore++;
        // reset ball
        currentX = botCursorValue * 2;
        currentY = 1;
        ballDX = 0;
        ballDY = -ballDY; // reverse direction
        displayGame(cursorValue, botCursorValue);
        delay(1500);
        return;
    }

  //bottom wall (player goal, bot scored)
    if (nextY > 3) {
        botScore++;
        // reset ball

        currentX = (cursorValue*2); //start ball where bot is
        currentY = 3;
        ballDX = 0;
        ballDY = -ballDY;
        displayGame(cursorValue, botCursorValue);
        delay(1500);
        return;
    }

        // player paddle collision
    // Paddle row is Y=3 (bottom). Paddle spans:
    // X = paddlePos*2 and X = paddlePos*2 + 1
    if (nextY == 3) {
        if (nextX/2 == cursorValue) {
            ballDY = -1; // bounce upward

            // Add deflection based on hit angle
```

```
            // if (ballDX == 0){}
             if (nextX % 2 == 0)      ballDX = -1; // left half of paddle
             else                     ballDX = 1;  // right half
          }
       }

       // bot paddle collision
       // Bot paddle at Y=0, X=4
       if (nextY == 0) {
          if (nextX/2 == botCursorValue) {
             ballDY = 1; // bounce downward
             if (nextX % 2 == 0) ballDX = -1;
             else ballDX = 1;
          }
       }

       // Apply final movement
       currentX = nextX;
       currentY = nextY;

       }
    void handleBot(int dif) {
       timeInterval++;

       int threshold = 20; // default for Easy

       switch(dif) {
          case 1: // Easy
             threshold = 20;  // moves slow, simple back/forth
             break;
          case 2: // Medium
             threshold = 10;  // moves faster
             break;
          case 3: // Hard
             threshold = 3;   // very fast
             break;
       }

       if(timeInterval >= threshold) {
          timeInterval = 0;

          int nextPaddlePos = botCursorValue;

          if(dif == 1) {
```

```
        // Easy: simple back/forth bounce
        nextPaddlePos += botPD;
        if(nextPaddlePos < 0 || nextPaddlePos > 7) {
            botPD = -botPD;
            nextPaddlePos = botCursorValue + botPD;
        }
    } else if(dif == 2) {
        // Medium: partially track the ball
        int ballColumn = currentX / 2;  // map ball X to column
        if(ballColumn < botCursorValue && random(0, 2)) nextPaddlePos--;  // 50% chance to
move
        else if(ballColumn > botCursorValue && random(0, 2)) nextPaddlePos++; // 50% chance

        // keep paddle in bounds
        if(nextPaddlePos < 0) nextPaddlePos = 0;
        if(nextPaddlePos > 7) nextPaddlePos = 7;
    } else {
        // Hard: always track the ball
        int ballColumn = currentX / 2;
        if(ballColumn < botCursorValue) nextPaddlePos--;
        else if(ballColumn > botCursorValue) nextPaddlePos++;

        // keep paddle in bounds
        if(nextPaddlePos < 0) nextPaddlePos = 0;
        if(nextPaddlePos > 7) nextPaddlePos = 7;
    }

    botCursorValue = nextPaddlePos;
  }
}



void handleRestart(){
  lcd.clear();
  playerScore = 0;
  botScore = 0;
}

void handleWin(int winner){
  handleRestart();
  lcd.setCursor(0, 0);
  if(winner == 1){
    lcd.print("YOU WIN!");
```

```
  }
  else
  {
    lcd.print("YOU LOSE!");
  }
  delay(5000); //wait 5 seconds before restarting: possible later change - flash continue? on
screen until player presses button

}

void handleExit(){
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("EXITING...");
  delay(3000);
}

// pick custom char index based on half-cell coordinates
int drawBall(){


  int lor = currentX % 2;   // left or right half of character
  int tob = currentY % 2;   // top or bottom half of character
  int chosenTile = TILE_BALL_TOPL;
  if(lor == 0 && tob == 0) chosenTile = TILE_BALL_TOPL;
  else if(lor == 0 && tob == 1) chosenTile = TILE_BALL_BOTL;
  else if(lor == 1 && tob == 0) chosenTile = TILE_BALL_TOPR;
  else chosenTile = TILE_BALL_BOTR;

  // load into CGRAM if needed (minimal)
  loadTile(chosenTile);
  int slot = tileSlot(chosenTile);
  if (slot < 0) slot = 0;
  return slot;
}

int drawBallandPaddle(int player){


  int lor = currentX % 2;   // left or right half of character
  int tob = currentY % 2;   // top or bottom half of character
```

```
  if(player == 1){ //bottom paddle
    int chosenTile = TILE_BOTTOM_P_BOTL;
    if(lor == 0 && tob == 0) chosenTile = TILE_BOTTOM_P_TOPL;
    else if(lor == 0 && tob == 1) chosenTile = TILE_BOTTOM_P_BOTL;
    else if(lor == 1 && tob == 0) chosenTile = TILE_BOTTOM_P_TOPR;
    else chosenTile = TILE_BOTTOM_P_BOTR;

    // load the overlap tile into CGRAM using the loader
    loadTile(chosenTile);
    int slot = tileSlot(chosenTile);
    if (slot < 0) slot = 4;
    return slot;
  }
  else if(player == 2){ //top paddle
    int chosenTile = TILE_TOP_P_BOTL;
    if(lor == 0 && tob == 0) chosenTile = TILE_TOP_P_TOPL;
    else if(lor == 0 && tob == 1) chosenTile = TILE_TOP_P_BOTL;
    else if(lor == 1 && tob == 0) chosenTile = TILE_TOP_P_TOPR;
    else chosenTile = TILE_TOP_P_BOTR;

    // load tile
    loadTile(chosenTile);
    int slot = tileSlot(chosenTile);
    if (slot < 0) slot = 4;
    return slot;
}
}

// ----------------- Display functions -----------------
void displayStart(){
  lcd.clear();
  lcd.setCursor(0, 0);

  //welcome:

  lcd.print("PRESS BUTTON");
  lcd.setCursor(0, 1);
  lcd.print("TO START");

}

void displayDifficulty(int mode){

  lcd.clear();
```

```
  lcd.print("PICK DIFFICULTY:");
  lcd.setCursor(0, 1);
  if(mode == 1){

  lcd.print("EASY ");
  lcd.write(127);
  }
  else if(mode == 2){

   lcd.print("MEDIUM ");
   lcd.write(127);
  }
  else {
    lcd.print("HARD ");
    lcd.write(127);
  }
}

void displayPause(int option){
  lcd.clear();
  lcd.print("GAME PAUSED ||");

  lcd.setCursor(0, 1);
  if(option == 1){

   lcd.print("RESUME ");
   lcd.write(127);
  }
  else {
    lcd.print("EXIT ");
    lcd.write(127);
}
}

void displayGame(int p1, int p2){
  // Render entire scene (simple)
  lcd.clear();

  // p2 paddle (top)


  // wall
  // ensure wall tile is available
```

```
loadTile(TILE_GAMEWALL);
lcd.setCursor(8, 0);
lcd.write((uint8_t)tileSlot(TILE_GAMEWALL));
lcd.setCursor(8, 1);
lcd.write((uint8_t)tileSlot(TILE_GAMEWALL));

// p1 paddle (bottom) at p1 column
// ensure paddle tiles are loaded
loadTile(TILE_PADDLE_BOTTOM);
loadTile(TILE_PADDLE_TOP);

// scores
lcd.setCursor(10, 0);
lcd.print("BOT:");
lcd.print(botScore);
lcd.setCursor(10, 1);
lcd.print("YOU:");
lcd.print(playerScore);

// ball
int bx = currentX / 2;
int by = currentY / 2;
if (bx < 0) bx = 0;
if (bx > 15) bx = 15;
if (by < 0) by = 0;
if (by > 1) by = 1;

if(bx == p1 && by == 1) //ball and player paddle in same square
{
  lcd.setCursor(bx, by);
  lcd.write(byte(drawBallandPaddle(1))); //ball and paddle

  lcd.setCursor(p2, 0); //other paddle
  lcd.write((uint8_t)tileSlot(TILE_PADDLE_TOP));
}
else if(bx == p2 && by == 0){ //ball and bot paddle in same square
  lcd.setCursor(bx, by);
  lcd.write(byte(drawBallandPaddle(2))); //ball and paddle


  lcd.setCursor(p1, 1); //other players paddles
  lcd.write((uint8_t)tileSlot(TILE_PADDLE_BOTTOM));

}
```

```
  else{
  lcd.setCursor(bx, by);
  lcd.write(byte(drawBall()));

  lcd.setCursor(p1, 1);
  lcd.write((uint8_t)tileSlot(TILE_PADDLE_BOTTOM));

  lcd.setCursor(p2, 0);
  lcd.write((uint8_t)tileSlot(TILE_PADDLE_TOP));
  }
}
```